

Mandatory Assignment 1

Alexander Fredbo-Nielsen and Emil Kjeldgaard Leth

March 08, 2026

Efficient portfolios and estimation uncertainty

Exercise 1.1

	monthly__mean	monthly__var	annualized__mean	annualized__var	annualized__vol
S&P500	0.0045	0.0019	0.0545	0.0228	0.1511

Exercise 1.2

```
# Exercise 2: Build the "true" parameters (mu, Sigma) for N=50 synthetic assets
```

```
N = 50
```

```
# Draw factor loadings and idiosyncratic volatilities (monthly units)
```

```
beta = np.random.uniform(0.5, 1.5, size=N)
```

```
sigma_eps = np.random.uniform(0.10/np.sqrt(12), 0.30/np.sqrt(12), size=N)
```

```
# True expected excess returns (monthly): mu_i = beta_i * mu_m
```

```
mu = beta * mu_m # shape (N,)
```

```
# True covariance matrix (monthly): Sigma = sigma_m^2 * beta beta' + diag(sigma_eps^2)
```

```
Sigma = sigma2_m * np.outer(beta, beta) + np.diag(sigma_eps**2) # shape (N, N)
```

```
# (Optional) quick sanity checks
```

```
print(mu.shape, Sigma.shape)
```

```
print(np.all(np.linalg.eigvals(Sigma) > 0)) # should be True (positive definite) if all s
```

```
(50,) (50, 50)
```

```
True
```

Exercise 1.3

```
import matplotlib.pyplot as plt

def mv_frontier_and_tangency(mu, Sigma, n_points=50):
    """
    Mean-variance efficient frontier + tangency portfolio (excess returns).
    Inputs:
        mu      : (N,) expected excess returns
        Sigma   : (N,N) covariance matrix of excess returns
    Returns:
        frontier_df : DataFrame with target_return and target_vol
        w_tan       : (N,) tangency weights (sum to 1)
    """
    mu = np.asarray(mu).reshape(-1)
    Sigma = np.asarray(Sigma)
    N = mu.shape[0]
    ones = np.ones(N)

    # Use linear solves (more stable than explicit inverse)
    Sinv_ones = np.linalg.solve(Sigma, ones)
    Sinv_mu = np.linalg.solve(Sigma, mu)

    # Scalar quantities for closed-form frontier
    A = ones @ Sinv_ones
    B = ones @ Sinv_mu
    C = mu @ Sinv_mu
    D = A * C - B**2

    # Target returns grid (within the spanned range of mu)
    R_min, R_max = float(mu.min()), float(mu.max())
    R_grid = np.linspace(R_min, R_max, n_points)

    # Frontier variance formula:  $\text{Var}(R) = (A R^2 - 2 B R + C) / D$ 
    var_grid = (A * R_grid**2 - 2 * B * R_grid + C) / D
    vol_grid = np.sqrt(np.maximum(var_grid, 0.0))

    frontier_df = pd.DataFrame({
        "target_return": R_grid,
        "target_vol": vol_grid
    })

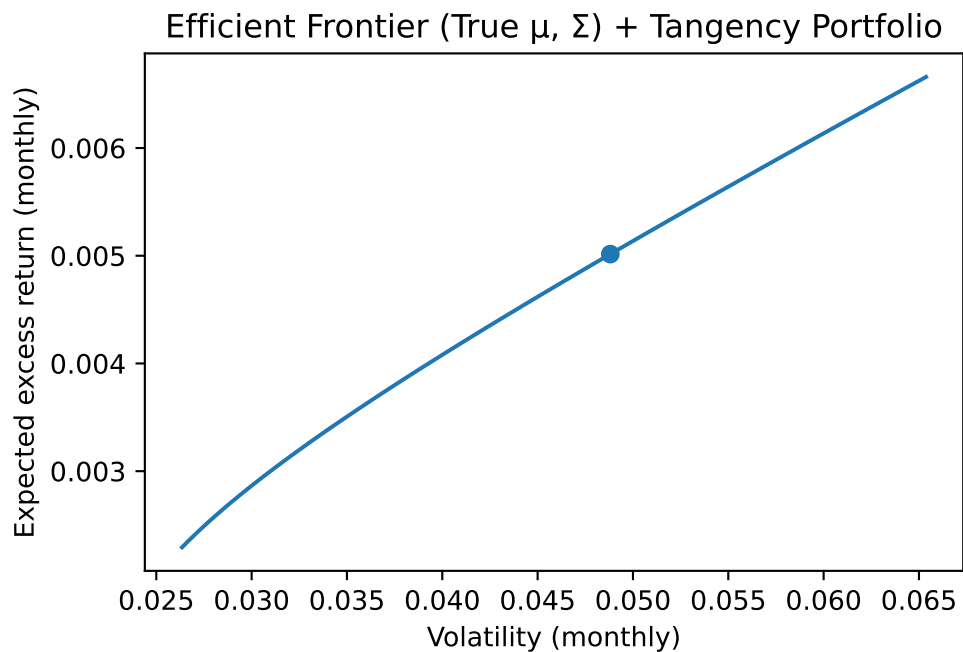
    # Tangency portfolio for excess returns:  $w = \Sigma^{-1} \mu$ , normalized to sum to 1
    w_tan_raw = Sinv_mu
    w_tan = w_tan_raw / (ones @ w_tan_raw)

    return frontier_df, w_tan
```

```
# --- Exercise 3: compute frontier + tangency using TRUE parameters (mu, Sigma) ---
frontier_df, w_tan = mv_frontier_and_tangency(mu, Sigma, n_points=80)

# Tangency portfolio moments (monthly excess)
mu_tan = float(w_tan @ mu)
vol_tan = float(np.sqrt(w_tan @ Sigma @ w_tan))
```

Exercise 1.4



```
0    0.006131
1    0.022468
2    0.006234
3    0.014955
4    0.013497
5    0.008352
6    0.021529
7    0.017308
8    0.046584
9    0.027645
Name: w_tan, dtype: float64
```

Exercise 1.5

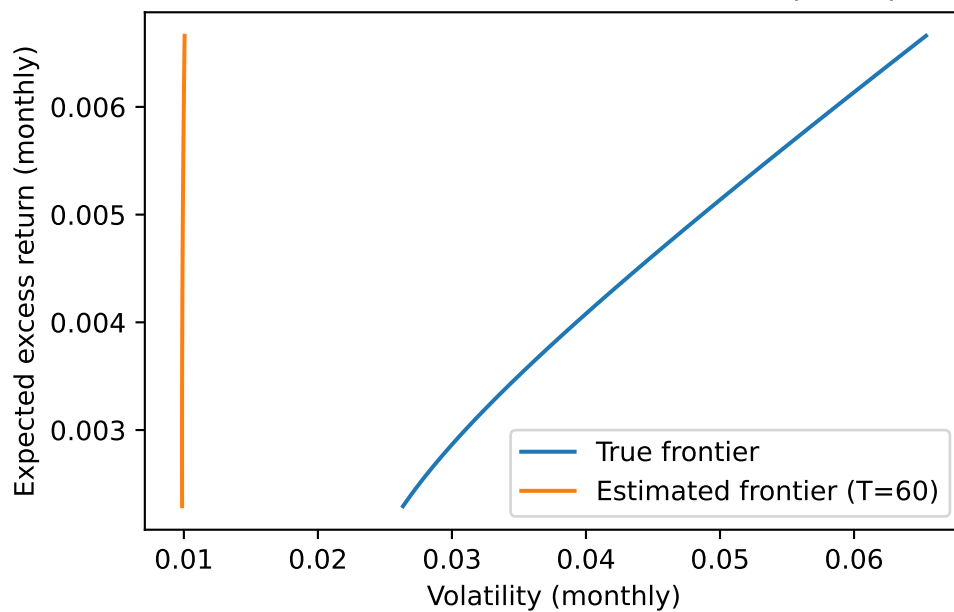
```
def simulate_returns(periods, expected_returns, covariance_matrix):
    # Draw T x N matrix of returns from  $N(\cdot, \Sigma)$ 
```

```

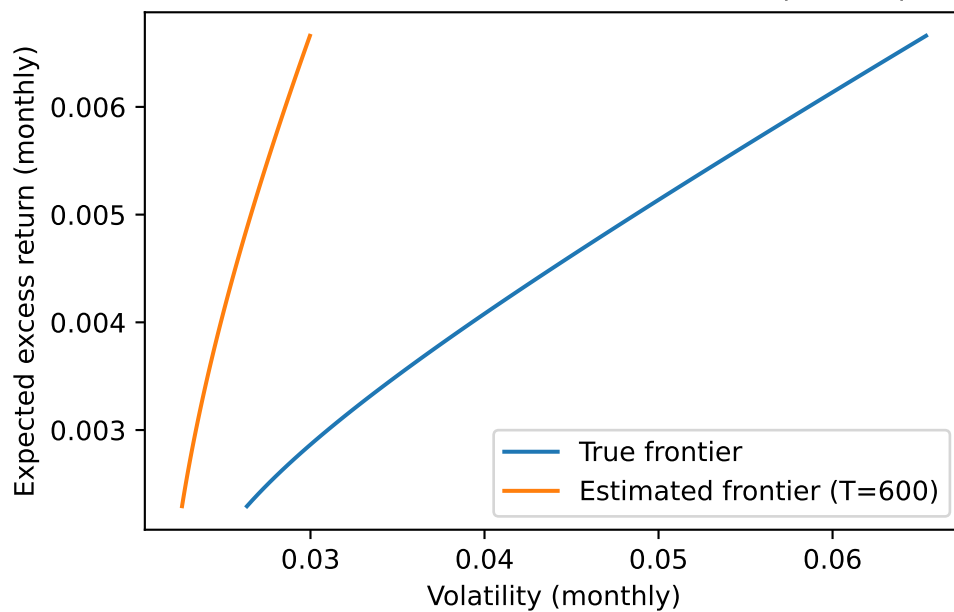
returns = np.random.multivariate_normal(
    mean=np.asarray(expected_returns).reshape(-1),
    cov=np.asarray(covariance_matrix),
    size=int(periods)
)
return returns

```

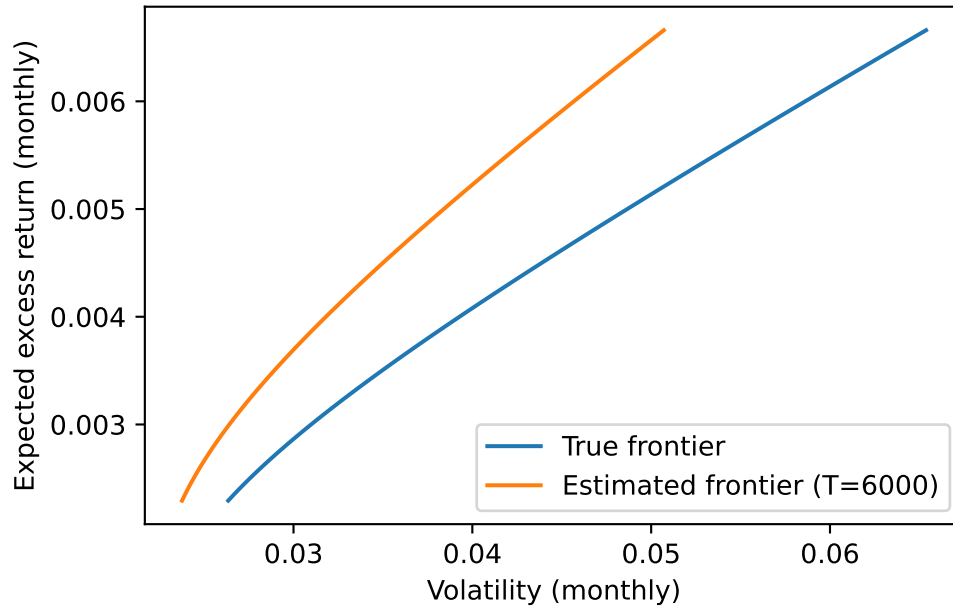
Efficient Frontier: True vs Estimated (T=60)



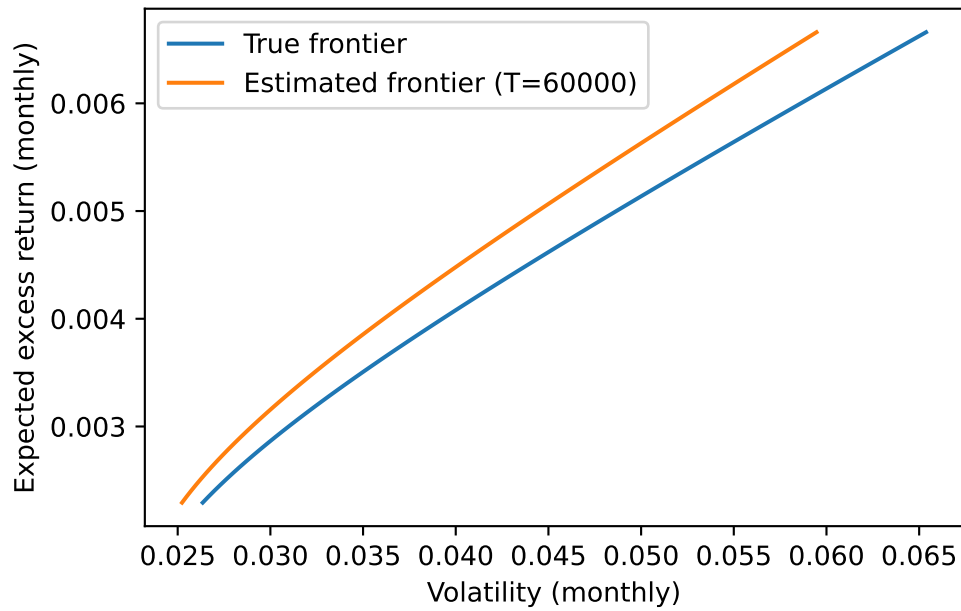
Efficient Frontier: True vs Estimated (T=600)



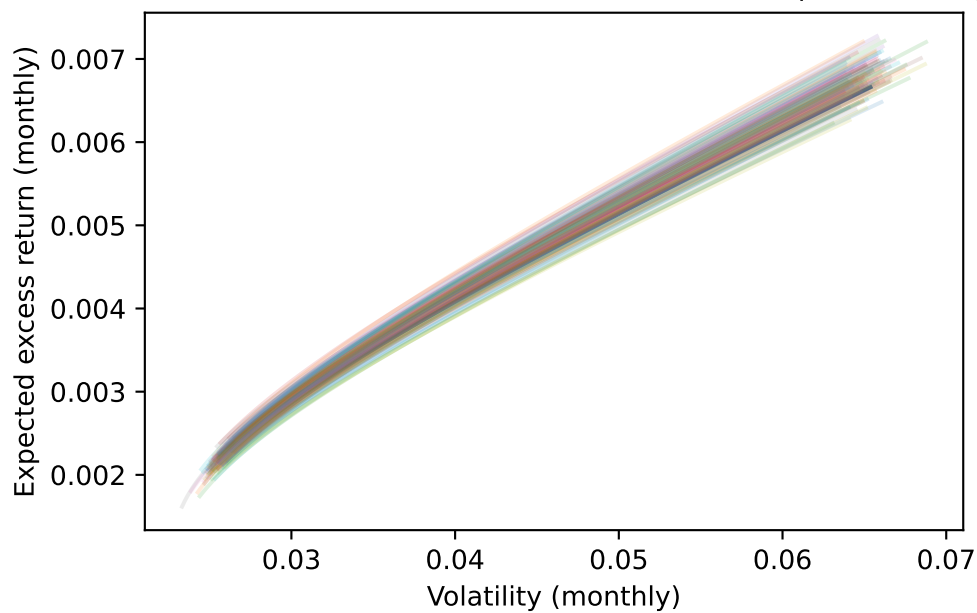
Efficient Frontier: True vs Estimated (T=6000)



Efficient Frontier: True vs Estimated (T=60000)

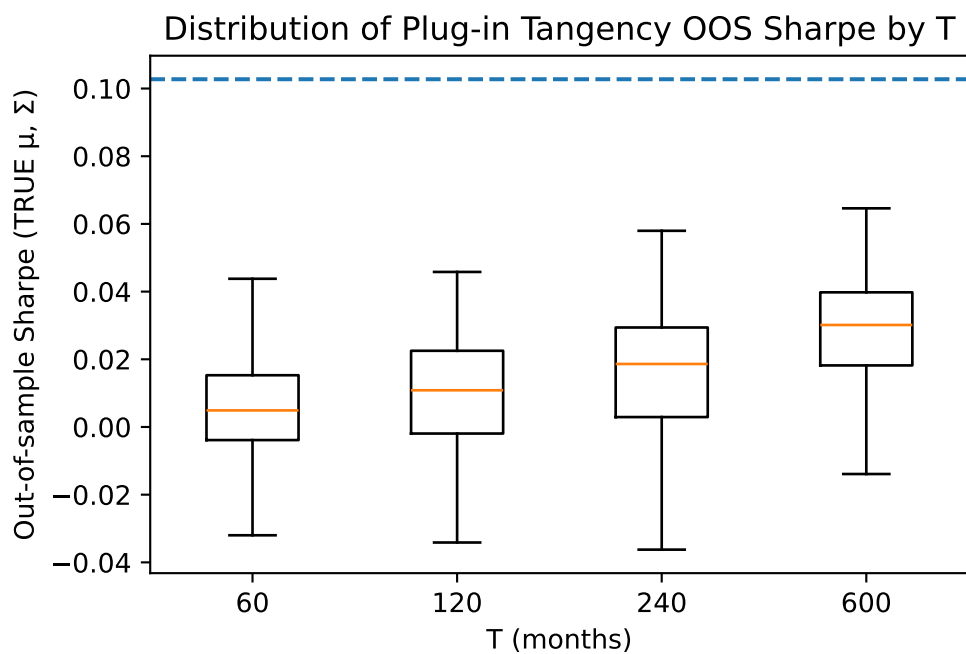
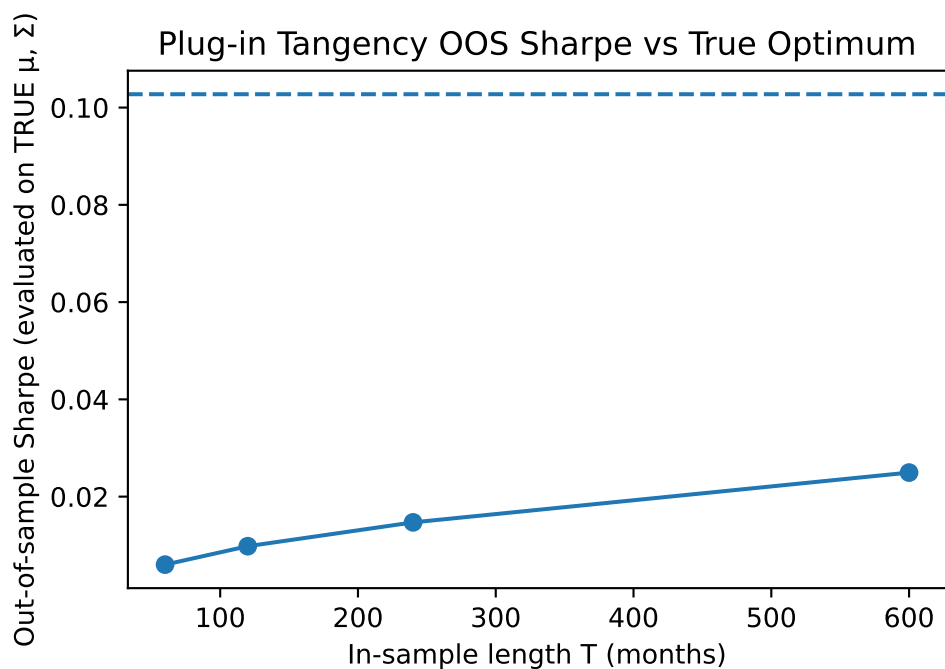


Estimation Error: 100 Estimated Frontiers (T=100000)



1.6

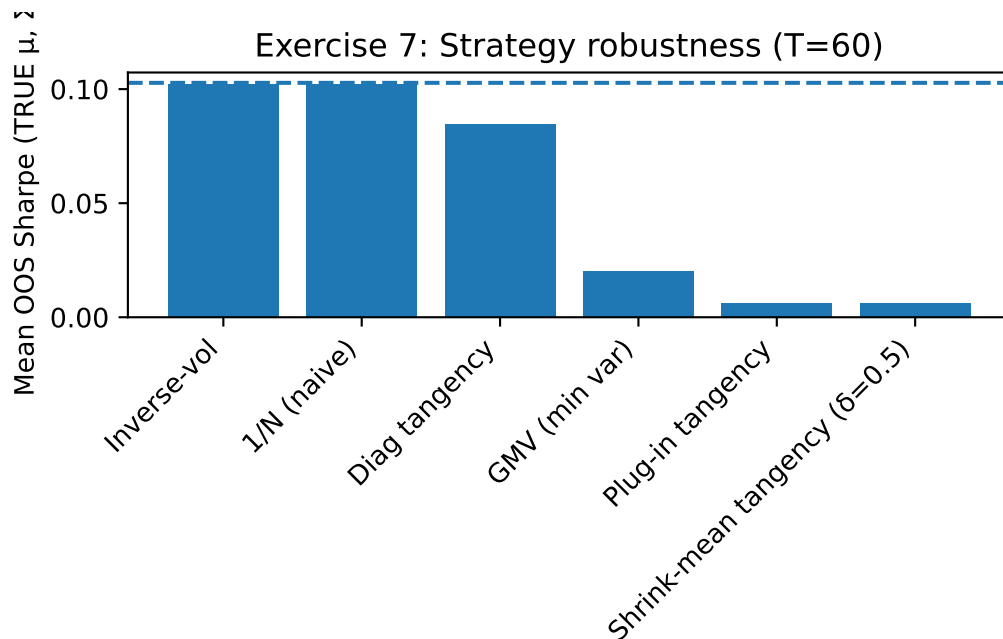
T	sharpe_oos			mean_oos			vol_oos		
	mean	std	median	mean	std	median	mean	std	median
60	0.0060	0.0140	0.0049	0.0002	0.0722	0.0012	0.9516	4.0149	0.2319
120	0.0098	0.0171	0.0108	0.0003	0.0260	0.0020	0.5883	1.5770	0.2169
240	0.0147	0.0200	0.0186	0.0002	0.0832	0.0029	0.7269	3.3413	0.2181
600	0.0249	0.0236	0.0301	0.0036	0.0228	0.0038	0.3259	0.6637	0.1470

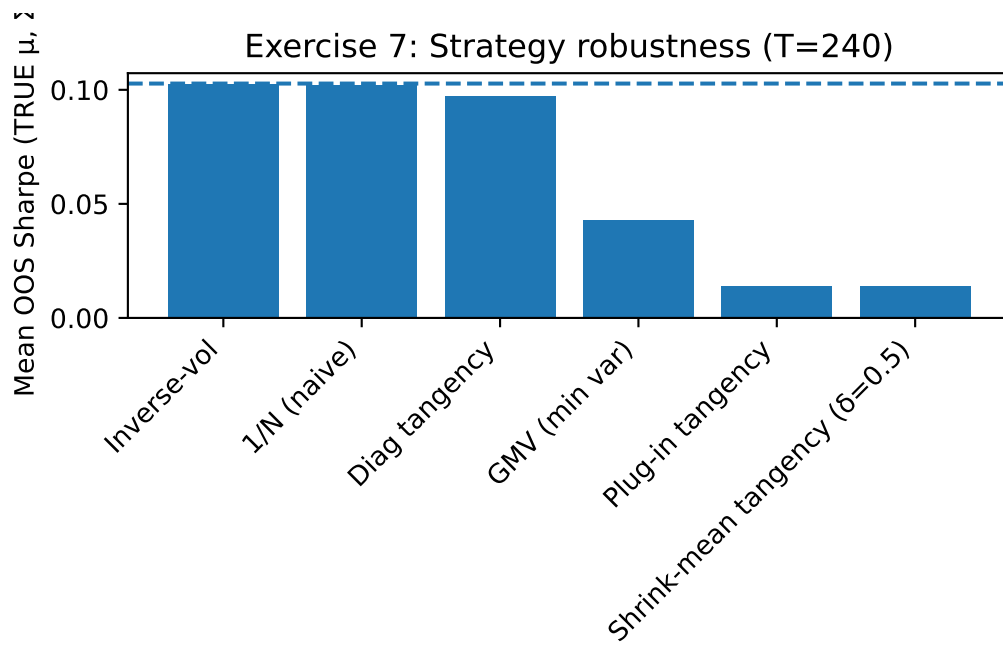
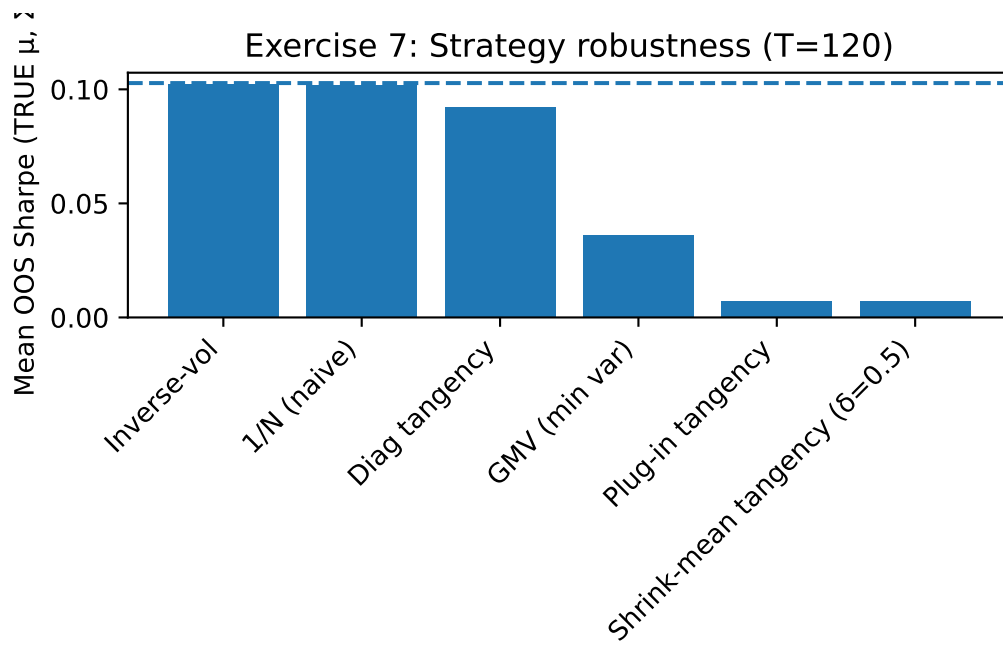


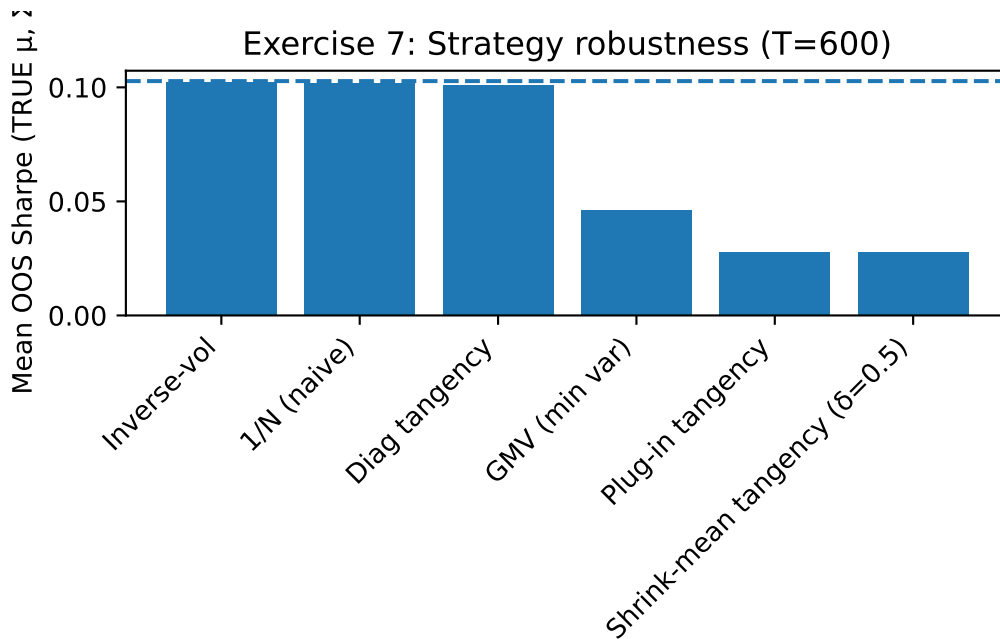
1.7

	T	strategy	mean	std	median
3	60	Inverse-vol	0.1022	0.0000	0.1022
0	60	1/N (naive)	0.1020	0.0000	0.1020
1	60	Diag tangency	0.0846	0.0283	0.0985
2	60	GMV (min var)	0.0202	0.0124	0.0201
4	60	Plug-in tangency	0.0059	0.0149	0.0061

	T	strategy	mean	std	median
5	60	Shrink-mean tangency ($\delta=0.5$)	0.0059	0.0149	0.0061
9	120	Inverse-vol	0.1022	0.0000	0.1022
6	120	1/N (naive)	0.1020	0.0000	0.1020
7	120	Diag tangency	0.0923	0.0215	0.1005
8	120	GMV (min var)	0.0361	0.0083	0.0363
10	120	Plug-in tangency	0.0070	0.0169	0.0074
11	120	Shrink-mean tangency ($\delta=0.5$)	0.0070	0.0169	0.0074
15	240	Inverse-vol	0.1022	0.0000	0.1022
12	240	1/N (naive)	0.1020	0.0000	0.1020
13	240	Diag tangency	0.0973	0.0130	0.1013
14	240	GMV (min var)	0.0426	0.0060	0.0434
16	240	Plug-in tangency	0.0138	0.0204	0.0160
17	240	Shrink-mean tangency ($\delta=0.5$)	0.0138	0.0204	0.0160
21	600	Inverse-vol	0.1022	0.0000	0.1022
18	600	1/N (naive)	0.1020	0.0000	0.1020
19	600	Diag tangency	0.1009	0.0069	0.1021
20	600	GMV (min var)	0.0464	0.0036	0.0465
22	600	Plug-in tangency	0.0279	0.0228	0.0334
23	600	Shrink-mean tangency ($\delta=0.5$)	0.0279	0.0228	0.0334







```

ticker = ["AAPL", "MSFT", "BA"]
prices = tf.download_data(
    domain="stock_prices",
    symbols=ticker,
    start_date="2000-01-01",
    end_date="2025-12-31"
)

returns = (prices
    .assign(ret=lambda x: x.groupby("symbol")["adjusted_close"].pct_change())
    .get(["symbol", "date", "ret"])
    .dropna(subset="ret")
)

```

1. Visualize the distribution of the daily returns.

Figure 1 shows the distribution of daily returns for the three assets.

1. Compute the sample mean μ and the variance-covariance matrix Σ of the returns. Which of the individual assets delivered the highest Sharpe ratio (assume the risk-free rate is zero) during the sample period?¹ The estimated moments may later be used for the construction of optimal portfolios (Markowitz (1952); Merton (1972))

Answer: The asset with the highest annualized Sharpe-ratio, computed as

$$\sqrt{250} \frac{\hat{\mu}_i}{\hat{\sigma}_i},$$

¹Whenever reporting performance measures, compute annualized values and clearly state how you calculated the measure.

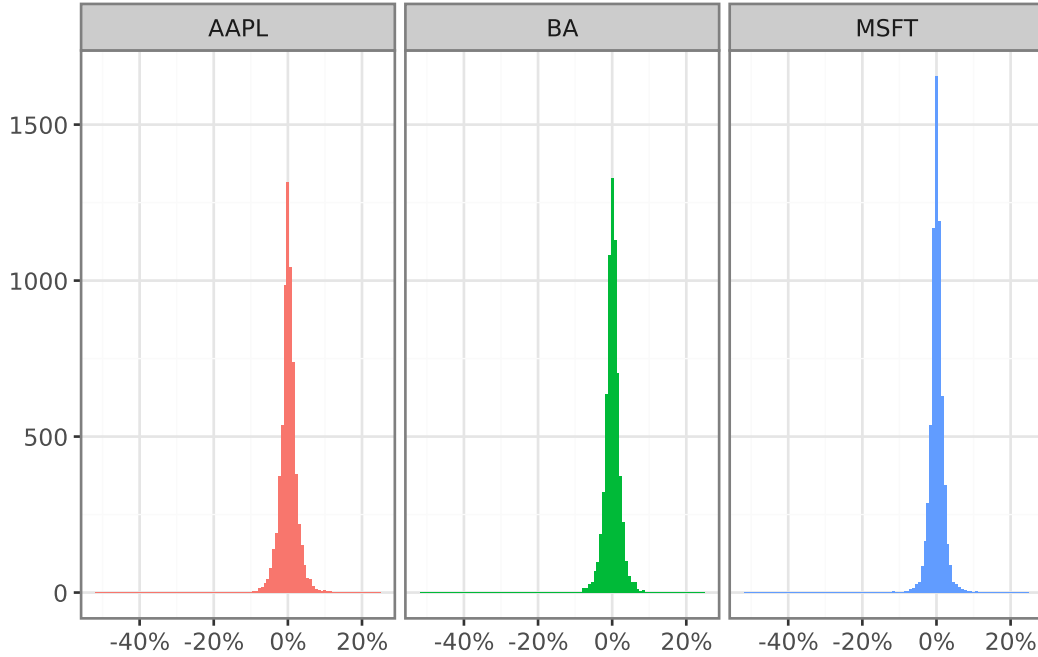


Figure 1: Distribution of daily stock returns.

Table 4: Historical annualized Sharpe-ratios

ticker	Sharpe
AAPL	0.776
MSFT	0.408
BA	0.484

where $\hat{\mu}_i$ is the sample mean of asset i 's daily returns and $\hat{\sigma}_i$ is the standard deviation of asset i 's daily returns. AAPL has an annualized Sharpe-ratio of `np.float64(0.776)`. Table 4 illustrates all Sharpe ratios.

References

- Markowitz, H. (1952). Portfolio selection. *The Journal of Finance*, 7(1):77–91.
- Merton, R. C. (1972). An analytic derivation of the efficient portfolio frontier. *Journal of Financial and Quantitative Analysis*, 7(4):1851–1872.